Bashir Agro Admin Panel

Dynamic Laravel + PostgreSQL Implementation Guide

Current scope: Build only the web Admin Panel and connect it to PostgreSQL. Do not implement the Flutter/farmer REST API yet. Preserve the supplied Bengali green agricultural UI.

1. Recommended Stack

• **Backend:** Laravel

• **Frontend:** Laravel Blade + existing HTML/CSS + Vanilla JavaScript

• **Database:** PostgreSQL

• **Later:** PostGIS for authoritative geofence calculations and REST API for the Flutter farmer app

2. What 'Dynamic' Means

The current prototype uses hard-coded JavaScript data and fixed dashboard numbers. The dynamic version must make PostgreSQL the source of truth. JavaScript remains only for UI interactions.

```
Admin -> Blade form -> Laravel route -> Controller -> Validation
      -> Eloquent/Service -> PostgreSQL -> Controller -> Blade -> Updated UI
```

Examples of data that must move out of JavaScript:

• Farmer records and farmer counts

• Task templates and assignments

• Farm locations

• Dashboard KPIs

• Report statistics

• Notification records

3. Convert index.html into Laravel Blade

```
resources/
  views/
    layouts/admin.blade.php
    components/
      sidebar.blade.php
      topbar.blade.php
      status-pill.blade.php
      modal.blade.php
      flash-message.blade.php
    dashboard/index.blade.php
    farmers/index.blade.php
    farmers/create.blade.php
    farmers/edit.blade.php
    farmers/show.blade.php
    locations/index.blade.php
    locations/create.blade.php
    locations/edit.blade.php
    locations/show.blade.php
    tasks/index.blade.php
    tasks/create.blade.php
    tasks/edit.blade.php
    tasks/show.blade.php
    assignments/index.blade.php
    assignments/create.blade.php
    assignments/show.blade.php
    monitoring/index.blade.php
    reports/index.blade.php
    notifications/index.blade.php
    settings/index.blade.php
```

```
css/admin.css
js/admin.js
```

Keep the visual design: Bengali labels, green sidebar, topbar, cards, status pills, tables, forms, modals and responsive behavior.

4. PostgreSQL Tables

#	Table	Purpose
1	users	Owner/Admin authentication
2	farmers	Main farmer records
3	farmer_emergency_contacts	Emergency contact for farmers
4	farm_locations	Fields, ponds, farms and coordinates
5	farmer_location	Farmer-location many-to-many link
6	task_templates	Reusable task definitions/rules
7	task_assignments	Assign task + farmer + location + schedule
8	task_submissions	Prepared for later farmer submissions
9	evidence_photos	Prepared for later evidence uploads
10	notifications	Persistent system/Admin notifications
11	settings	Mutable Admin defaults
12	activity_logs	Audit trail of Admin changes

5. users

```
id BIGINT PK
name VARCHAR(150)
email VARCHAR(255) UNIQUE NULL
phone VARCHAR(20) UNIQUE NULL
password VARCHAR(255)
role VARCHAR(30)        -- owner/admin
is_active BOOLEAN DEFAULT true
last_login_at TIMESTAMP NULL
remember_token VARCHAR NULL
created_at
updated_at
```

6. farmers

```
id BIGINT PK
farmer_code VARCHAR(30) UNIQUE
full_name VARCHAR(150)
phone VARCHAR(20) UNIQUE
nid VARCHAR(30) NULL
date_of_birth DATE NULL
gender VARCHAR(20)
address TEXT
primary_activity VARCHAR(100)
experience_years INTEGER NULL
profile_photo_path VARCHAR(500) NULL
app_access_enabled BOOLEAN DEFAULT true
status VARCHAR(30) DEFAULT 'active'
created_by BIGINT FK -> users.id
created_at
updated_at
deleted_at
```

7. farmer_emergency_contacts

```
id BIGINT PK
farmer_id BIGINT FK -> farmers.id
name VARCHAR(150)
phone VARCHAR(20)
relationship VARCHAR(50)
created_at
updated_at
```

8. farm_locations

```
id BIGINT PK
location_code VARCHAR(30) UNIQUE
name VARCHAR(150)
type VARCHAR(50)
address TEXT
area DECIMAL NULL
area_unit VARCHAR(30) NULL
latitude DECIMAL NULL
longitude DECIMAL NULL
geofence_radius_meters INTEGER DEFAULT 20
status VARCHAR(30) DEFAULT 'active'
created_by BIGINT FK -> users.id
created_at
updated_at
deleted_at
```

9. farmer_location Pivot

```
id BIGINT PK
farmer_id BIGINT FK -> farmers.id
farm_location_id BIGINT FK -> farm_locations.id
created_at
UNIQUE(farmer_id, farm_location_id)
```

10. task_templates

```
id BIGINT PK
task_code VARCHAR(30) UNIQUE
name VARCHAR(200)
category VARCHAR(100)
description TEXT NULL
instructions TEXT
required_photo_count INTEGER DEFAULT 0
require_realtime_photo BOOLEAN DEFAULT true
allow_gallery_upload BOOLEAN DEFAULT false
require_geofence BOOLEAN DEFAULT true
recurrence_type VARCHAR(30)
is_active BOOLEAN DEFAULT true
created_by BIGINT FK -> users.id
created_at
updated_at
deleted_at
```

11. task_assignments

```
id BIGINT PK
task_template_id BIGINT FK
farmer_id BIGINT FK
farm_location_id BIGINT FK
scheduled_date DATE
start_time TIME
end_time TIME NULL
priority VARCHAR(30)
status VARCHAR(30)
assigned_by BIGINT FK -> users.id
assigned_at TIMESTAMP
started_at TIMESTAMP NULL
completed_at TIMESTAMP NULL
created_at
updated_at
```

12. task_submissions - Prepare for Later

```
id BIGINT PK
task_assignment_id BIGINT FK
farmer_id BIGINT FK
submitted_at TIMESTAMP
latitude DECIMAL NULL
longitude DECIMAL NULL
geofence_passed BOOLEAN NULL
distance_meters DECIMAL NULL
is_late BOOLEAN DEFAULT false
validation_status VARCHAR(50)
farmer_note TEXT NULL
created_at
updated_at
```

13. evidence_photos - Prepare for Later

```
id BIGINT PK
task_submission_id BIGINT FK
file_path VARCHAR(500)
source VARCHAR(30)
captured_at TIMESTAMP NULL
latitude DECIMAL NULL
longitude DECIMAL NULL
created_at
updated_at
```

14. settings

```
id BIGINT PK
key VARCHAR(150) UNIQUE
value TEXT
type VARCHAR(30)
updated_by BIGINT FK -> users.id NULL
created_at
updated_at
```

15. activity_logs

```
id BIGINT PK
user_id BIGINT FK -> users.id
action VARCHAR(100)
subject_type VARCHAR(150)
subject_id BIGINT NULL
description TEXT NULL
old_values JSONB NULL
new_values JSONB NULL
ip_address VARCHAR(45) NULL
created_at
```

16. Main Eloquent Relationships

```
Farmer hasOne FarmerEmergencyContact
Farmer belongsToMany FarmLocation
FarmLocation belongsToMany Farmer

Farmer hasMany TaskAssignment
TaskAssignment belongsTo Farmer
TaskAssignment belongsTo TaskTemplate
TaskAssignment belongsTo FarmLocation

TaskTemplate hasMany TaskAssignment
FarmLocation hasMany TaskAssignment
```

One farmer can have many task assignments. Each individual task assignment belongs to exactly one farmer.

17. Laravel Models

```
app/Models/
  User.php
  Farmer.php
  FarmerEmergencyContact.php
  FarmLocation.php
  TaskTemplate.php
  TaskAssignment.php
  TaskSubmission.php
  EvidencePhoto.php
  Setting.php
  ActivityLog.php
```

18. Admin Controllers

```
app/Http/Controllers/Admin/
  DashboardController.php
  FarmerController.php
  FarmLocationController.php
  TaskTemplateController.php
  TaskAssignmentController.php
  MonitoringController.php
  ReportController.php
  NotificationController.php
  SettingController.php
```

19. Web Routes Only for This Stage

```
Route::prefix('admin')
  ->name('admin.')
  ->middleware(['auth'])
  ->group(function () {
      Route::get('/dashboard', ...);
      Route::resource('farmers', FarmerController::class);
      Route::resource('locations', FarmLocationController::class);
      Route::resource('tasks', TaskTemplateController::class);
      Route::resource('assignments', TaskAssignmentController::class);
      Route::get('/monitoring', ...);
      Route::get('/reports', ...);
      Route::get('/notifications', ...);
      Route::get('/settings', ...);
  });
```

Leave routes/api.php alone for now. The farmer REST API will be integrated later.

20. Make the Dashboard Dynamic

Dashboard KPIs must be calculated from PostgreSQL rather than fixed numbers.

```
$activeFarmers = Farmer::where('status', 'active')->count();

$todaysTasks = TaskAssignment::whereDate(
    'scheduled_date',
    today()
)->count();
```

```
$activeLocations = FarmLocation::where(
    'status',
    'active'
)->count();
```

Today's assignment table should eager-load farmer, task template and location to avoid unnecessary queries.

21. Farmer Management Requirements

• Create, read, update and soft delete

• Search by farmer name, farmer code and phone

• Filter active/inactive

• Pagination

• Emergency contact

• Server-generated farmer code such as F-000001

• Validation and flash messages

22. Farm Location Requirements

• Create, read, update and soft delete

• Search and active/inactive state

• Area and area unit

• Latitude and longitude

• Default geofence radius of 20 meters

• Server-generated location code

• Keep the current fake map as a temporary visual placeholder if needed; add a real map later

23. Task Template Requirements

• Create, read, update and soft delete

• Search and active/inactive state

• Required photo count

• Real-time photo flag

• Gallery allowed flag

• Geofence required flag

• Recurrence and instructions

• Server-generated task code

24. Task Assignment Requirements

• Select an existing active farmer from PostgreSQL

• Select an existing active task template

• Select an existing active farm location

• Scheduled date, start time and end time

• Priority and status

• assigned_by and assigned_at

25. JavaScript's New Job

Keep JavaScript for presentation and interaction, not as a database.

• Sidebar opening/closing

• Modal behavior

• Confirmation dialogs

• Switch and photo-count UI

• Toast messages

• Future map interaction and asynchronous search

26. Form Requests

```
app/Http/Requests/
  StoreFarmerRequest.php
  UpdateFarmerRequest.php
  StoreLocationRequest.php
  UpdateLocationRequest.php
  StoreTaskTemplateRequest.php
  UpdateTaskTemplateRequest.php
  StoreAssignmentRequest.php
  UpdateSettingsRequest.php
```

Use @csrf, old() values, server-side validation errors and success/error flash messages in Blade forms.

27. Transactions and Indexes

Use DB::transaction() for multi-table operations such as farmer + emergency contact + location links + activity log.

```
Recommended indexes:
farmers: farmer_code, phone, status, primary_activity
farm_locations: location_code, status, type
task_templates: task_code, category, is_active
task_assignments: farmer_id, task_template_id,
                  farm_location_id, scheduled_date, status
```

28. Admin Panel Build Order

```
1. Laravel installation + PostgreSQL connection
2. Admin login + Admin layout
3. Convert index.html into Blade
4. Migrations + Models + Relationships
5. Farmer Management
6. Farm Location Management
7. Task Template Management
8. Task Assignment
9. Dynamic Dashboard
10. Monitoring page structure
11. Reports page structure
12. Notifications
13. Settings
14. Search + Filters + Pagination
15. Activity Logs
16. Testing

ADMIN PANEL COMPLETE

LATER:
Flutter + REST API + authoritative PostGIS geofence workflow
```

29. Master Antigravity Instruction

Use the following as the project instruction. Implement the first task only, verify it, then stop.

```
PROJECT: Bashir Agro Admin Panel

CURRENT SCOPE:
Build ONLY the dynamic web Admin Panel.

Do NOT implement the Flutter farmer API yet.
Do NOT implement mobile authentication yet.
Do NOT build external REST API endpoints yet.

TECH STACK:
Backend: Laravel
Frontend: Laravel Blade + HTML + CSS + Vanilla JavaScript
Database: PostgreSQL
Future: PostGIS + Flutter + REST API

I already have a completed HTML/CSS/JavaScript prototype.
Use my existing index.html as the visual source of truth.
Do NOT redesign the interface.

Preserve:
- Bengali interface
- dark-green sidebar
- topbar
- green agricultural theme
- dashboard cards
- responsive layout
- tables
- status pills
- forms
- modals
- buttons
- typography and spacing as closely as practical
- mobile responsive behavior

The existing JavaScript contains hard-coded demo data.
Convert the prototype into a real dynamic Laravel Admin Panel backed by PostgreSQL.

DATABASE DATA MUST COME FROM:
Laravel -> Eloquent -> PostgreSQL.

JavaScript should only handle UI interactions.

ADMIN MODULES:
1. Authentication
2. Dashboard
3. Farmer Management
4. Farm Location Management
5. Task Template Management
6. Task Assignment
7. Activity Monitoring
8. Reports
9. Notifications
10. Settings
11. Activity Logs

DATABASE TABLES:
users
farmers
farmer_emergency_contacts
farm_locations
farmer_location
task_templates
task_assignments
task_submissions
evidence_photos
notifications
settings
activity_logs

task_submissions and evidence_photos should be structurally planned for future integration,
but do not build the mobile/API submission workflow now.
```

```
RELATIONSHIPS:
Farmer has one emergency contact.
Farmer belongs to many farm locations.
Farm location belongs to many farmers.
Farmer has many task assignments.
Task template has many task assignments.
Farm location has many task assignments.
Task assignment belongs to exactly one farmer.
Task assignment belongs to exactly one task template.
Task assignment belongs to exactly one farm location.

REQUIREMENTS:
- Convert index.html to reusable Blade layout/components/pages.
- Move CSS to resources/css/admin.css.
- Move UI JavaScript to resources/js/admin.js.
- Remove hard-coded farmer/task/location/statistics data from JavaScript.
- Use Eloquent models and controllers.
- Use Form Requests for validation.
- Use foreign keys and appropriate indexes.
- Use soft deletes where appropriate.
- Use PostgreSQL JSONB for activity-log old/new values.
- Use DB transactions for multi-table operations.
- Generate farmer/location/task codes server-side.
- Dashboard values must come from PostgreSQL.
- Assignment dropdowns must load real farmers/tasks/locations.
- Add search, filters and pagination.
- Preserve the current UI.

SECURITY:
Use Laravel authentication, password hashing, CSRF, authorization,
validation, mass-assignment protection and safe file validation.

DO NOT:
- create React or Vue frontend
- introduce FastAPI
- create REST API yet
- redesign my interface
- keep hard-coded database records in JavaScript
- put business logic inside Blade
- put all logic in one controller
- query PostgreSQL directly from Blade
- trust frontend validation
- put credentials in source code

WORKING METHOD:
1. Inspect existing files.
2. Explain planned changes.
3. Modify only necessary files.
4. Preserve existing design.
5. Run migrations where needed.
6. Run tests.
7. Fix errors.
8. Report files changed.
9. Stop before the next major module unless instructed.

FIRST TASK ONLY:
1. Laravel project setup
2. PostgreSQL configuration
3. Admin authentication
4. Convert existing index.html layout into Laravel Blade
5. Create sidebar/topbar/layout components
6. Move CSS and JavaScript into proper asset files
7. Make Dashboard route work
8. Preserve the existing design

Do NOT create Farmer, Location, Task or Assignment modules yet.

When this first task works, STOP and report what was completed.
```

30. Important Scope Decision

For this Admin-only stage, normal PostgreSQL latitude/longitude fields are sufficient to make CRUD screens dynamic. When the farmer app/API is introduced and geofence distance/polygon checks become security-sensitive, use PostGIS as the authoritative spatial layer.